

SWS3026 Visual Computing Group Project

Real-time Video Analysis and Rendering

Part I. Environment Setup

This project must be run **locally** on your machine and cannot be executed in *Google Colab* or *Jupyter Notebook*, as it requires access to the laptop's camera, which is not supported in browser-based environments. You can set up your environment using either Anaconda or a standard Python installation with `pip`.

After setting up Python, install the required libraries: `opencv-python`, `scikit-learn`, `numpy`, `matplotlib`, `pillow` and `ultralytics`. These libraries are essential for computer vision, machine learning, and data visualization. For more information, refer to the official documentation for OpenCV, scikit-learn, NumPy, Matplotlib, Pillow and Ultralytics. Once the environment is ready, you can run the `.py` files directly from a terminal or supported IDE.

Part II. Beginner Level

The aim of this level is to introduce you to real-time facial keypoints detection using a webcam feed. You will gain hands-on experience with OpenCV, experiment with different keypoints detection techniques, and develop an understanding of their strengths and limitations. **Read all instructions first before coding. Some planning is required.**

Task 1: Webcam Activation & Real-Time Display

In this task, you will access the webcam and create a real-time display window using OpenCV's `cv2.VideoCapture()` function. `webcam.py` provides a minimal example to get you started. As shown in Figure 1, you'll eventually be able to activate your laptop's webcam and see the detection results displayed live on the screen. So make sure your webcam is working. If you've got tape over it for privacy, now's the time to set it free!

Task 2: Facial Keypoints Detection

In this task, you will explore an example that uses OpenCV's *Haar Cascade classifiers* to detect simple facial keypoints. The goal is to implement real-time face detection and estimate the locations of basic facial features, such as the eyes and mouth.

1. To start, use `cv2.CascadeClassifier('haarcascade_frontalface_default.xml')` to load a pre-trained face detection model. For each frame captured from the webcam, apply the classifier to detect faces in the image. Once faces are detected, draw rectangles around them to visualize the results in the live video feed. You can use `cv2.rectangle()` function to draw a bounding box around the detected face.
2. After detecting the face region, you will estimate and display basic keypoints, such as the eyes and mouth, within each detected bounding box. Specifically, you can use the *Local Binary Features (LBF)* model by initializing a facemark detector with `cv2.face.createFacemarkLBF()` and loading the pre-trained model file with `facemark.loadModel('lbfmodel.yaml')`. This model can predict 68 facial landmarks, including detailed positions of the eyes, nose, mouth, and jawline.
3. Finally, visualize the detected keypoints by drawing small circles or markers on the live video feed at the predicted landmark positions. This will help you verify the accuracy of



Figure 1: This is an illustration of Task 1 of the Beginner Level.

the keypoint detection in real time.

At the end of Task 2, you should be able to see the bounding box and detected keypoints, as shown in Fig. 2

Task 3: Is the Face Detector Robust?

In this task, you will evaluate the performance of your current face detection and keypoint visualization pipeline. Observe how the system behaves under different conditions, such as changes in lighting, head angles, and partial occlusions (e.g., cover your mouth with your hand). You should identify any challenges or limitations you encounter and reflect on the possible causes.

Based on your observations, suggest at least one way to improve the system. This could involve applying image preprocessing techniques, adjusting detection parameters, or exploring alternative models or libraries, such as Dlib and MediaPipe.

You are not required to try all of these alternatives, but we encourage you to experiment with at least one other face detector and compare its performance with the Haar cascade method. In your discussion, you need to explain why the alternative method may offer improved results.

Part III. Expert Level

In this level, you will use facial keypoint detection to classify facial expressions in real time. Rather than using the full image as input, you will rely on the detected facial keypoints. You will learn how to train a classifier, evaluate its performance, and enhance its accuracy.

Task 1: Facial Expression Classification

You will use a Facial Expression dataset to develop and improve your classifier for this task.

1. To start, find the dataset **FER-2013** in the **Expert Level** folder. Make sure to carefully review the information provided about the dataset. Split the dataset into a training and

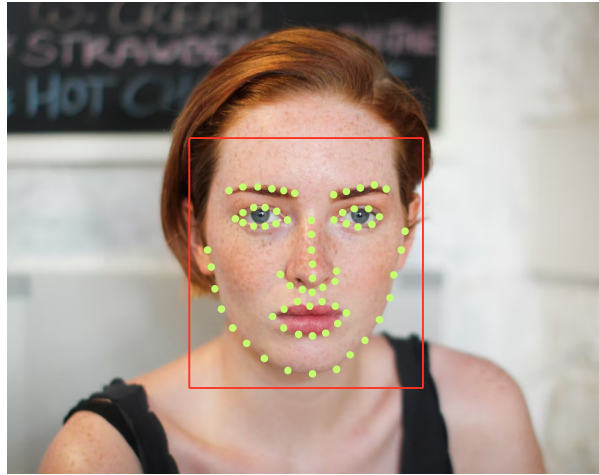


Figure 2: This is an example of facial keypoint detection using an image downloaded from the internet.

testing set (and validation set if necessary).

2. Use the facial keypoint detection method from **Part I** on each image to extract the keypoints. *You may apply any pre-processing to the keypoints – for instance, normalizing their positions or creating new images by plotting black circles representing the keypoints on a white background.*
3. Using the training set, train a classifier using the extracted keypoints instead of the full images. *You may choose either a machine learning approach (e.g., SVM, Random Forest) or a deep learning model (e.g., MLP, CNN).* However, ensure that your classifier can make predictions in real time – specifically, within 30 milliseconds per prediction to support 30 frames per second.
4. Did you face any **challenges** in classifying the facial expressions? What **key strategies or solutions** did you use to overcome these issues? Using the test set, evaluate and report your classification performance on the dataset by analyzing the confusion matrix and examining failure cases. You may present the results both **before and after** applying your proposed solution. Cross-validation is optional.

With your facial expression classifier, it's time to put it to use in a fun and creative way.

Task 2: Video Effects

In this task, you will apply your facial expression classifier to **webcam input** and design **video effects** based on the predicted facial expression classes. Figure 3 illustrates examples from Face Time, where different expressions trigger various video effects. For example, smiling can activate a playful sparkle overlay around the face, while raising the eyebrows might cause animated stars to appear above the head. These dynamic effects enhance user engagement by responding instantly to facial movements and emotions captured in the video feed. Feel free to be creative and develop your own effects. *To begin, however, we recommend starting with a simple approach, such as displaying a text box that shows the predicted facial expression using `cv2.putText` function.* You can even extend your classifier from Task 1 to recognize additional facial expressions (such as sticking out your tongue), thus enabling a wider range of effects. Remember to share your fun video effects with friends, classmates, and students

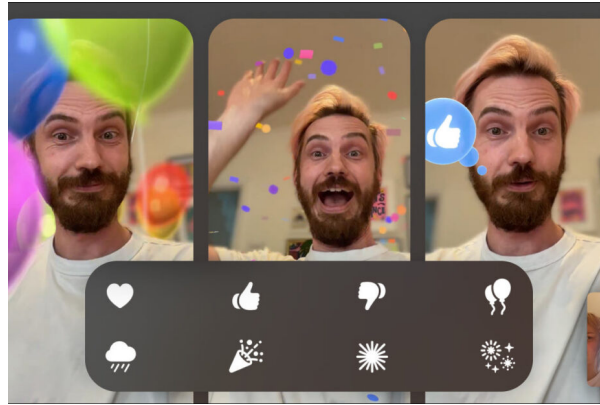
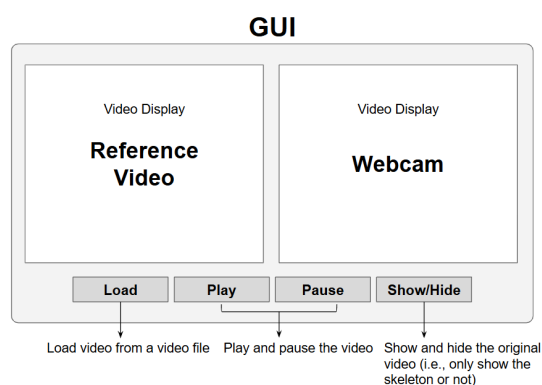


Figure 3: These are examples from Face Time, illustrating different video effects. You can be creative and design your own video effects.

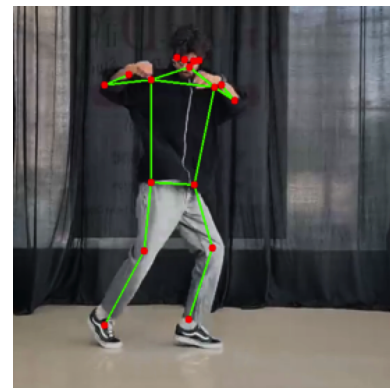
from other courses. Take advantage of **Poster/Demo Day** to **showcase your work and impress your audience!**

Part IV. Bonus Level

At this level, you will apply your previous knowledge to a more advanced application. Rather than focusing on facial keypoint tracking, you will now detect full- or partial-body movements in video from a file or from webcam input, and use those movements to create a fun, interactive experience. **Read all instructions first before coding. Some planning is required.**



(a) Illustration of the GUI created by running the example code, `danceapp.py`.



(b) Example of keypoint detection of body movements from a video.

Figure 4: Combined view of the GUI (left) and keypoint detection (right).

Task 1: TikTok Dance Analysis

For this task, you will work with a TikTok dance video dataset to build and refine your full or partial body movement detection pipeline.

1. To start, find the dataset **TikTok Dataset** in the **Bonus Level** folder. Carefully review the information provided about the dataset. *This dataset is primarily intended for exploring and enhancing your body movement detection pipeline. We do not expect this*



Figure 5: Just Dance by Ubisoft.

dataset to be used for training machine learning or deep learning models.

2. We provide a minimal example code, `danceapp.py`, to get you started. Running the code will launch a GUI, as shown in Figure 4a. For this task, you only need to use the left panel (i.e., the reference video). Additionally, the example code includes a sample keypoint detection method to help you get started. Figure 4b shows an example of keypoints detected and visualized in a video.
3. Did you face any **challenges** in detecting keypoints for full or partial body movements? What **key strategies or solutions** did you use to overcome these issues? *For example, if there are multiple people in the video, how do you determine who is dancing?* Report your findings (in the presentation). **Important Note:** *There is no need to evaluate your pipeline on the entire dataset. You can focus on a few examples for testing.*

With your body movement detection pipeline now up and running, it's time to put it to use in a fun and creative way.

Task 2: Just Dance

Just Dance (see Figure 5) is a rhythm game series created by Ubisoft, where players follow on-screen choreography to a variety of songs and are scored based on how closely they match the dancers' movements. Now it is your turn to build a similar dance application. Rather than using game controllers to track movement, you will use your body movement detection pipeline to **recognize and evaluate your dance moves directly from the webcam**.

1. To begin, apply the detection pipeline you developed in **Task 1** to both the reference video – the one you are imitating – and your webcam input. You can now make full use of both panels in the GUI, as illustrated in Figure 4a. Apply the detection pipeline and display the body movements in both panels. **Important Note:** *You are not required to use videos from the TikTok dataset. You are free to choose your own reference videos, including those from various dance genres.*
2. Create **your own scoring system** to evaluate your dance performance. For example, there may be a lag between the reference video and your dance from the webcam, how do you take this lag into account when designing your scoring system? Given two sets of keypoint positions, how do you align them (spatially and temporally) and compare their similarity? You may also analyze how well your movements match the rhythm of the music (e.g., you may use python libraries, `librosa` or `madmom`, to analyze the music). Feel free to explore additional metrics to make your evaluation more comprehensive. Do not forget to display the feedback to the user (e.g., showing “Perfect”, “Super”, “Good” on

the screen, similar to Figure 5). ***Important Note:*** *During the project presentation, you should explain techniques and metrics used in your scoring system.*

3. Your dance application **does not have to be limited to just dancing**. Why not make it more engaging by adding a bonus activity? For instance, you could incorporate interactive elements like pop-up fruits on the webcam that players must “slice” for extra points – combining ideas from games like Fruit Ninja and Just Dance. Do not hold back – feel free to get creative!

Now it is time to share your dance application with friends, classmates, and students from other courses. Take advantage of **Poster/Demo Day** to **showcase your work and impress your audience!**

Remarks

1. There is no compulsion to use any deep network for this group project. If you choose to do so, make sure your own laptop is powerful enough to train and run the deep network. We will not provide powerful GPUs for you to use.
2. Use Canvas Discussion to ask questions and seek help from other groups.

— END —